```
ob_start(function ($buffer) {
    return str_replace('Jess', 'Naughty Jess', $buffer);
});

print "A cat called Jess is begging for food again.\n";
print "This is forbidden.\n";
http_response_code(403);

flush();
```

```
$ curl -v http://localhost:8080/index.php
...
< HTTP/1.1 403 Forbidden
< Host: localhost:8080
< Date: Mon, 14 Jul 2025 18:19:36 GMT
< Connection: close
< X-Powered-By: PHP/8.4.6
< Content-type: text/html; charset=UTF-8
<
A cat called Naughty Jess is begging for food again.
This is forbidden.
```

It is even possible to have nested levels of buffering, and there are many other functions for manipulating the buffering mechanism. If you are interested in finding out more, you can start with the official documentation at https://www.php.net/manual/en/book.outcontrol.php

Using a Library

We can pause here for a moment. We've covered quite a bit of ground with HTTP and how PHP interacts with it. My aim here was to teach the fundamentals of the protocol in addition to the behavior and tools that PHP offers. In a real-world application, I would *strongly* advise you to consider using a library rather than rolling your own code. When I introduced the concept of URL routing, I recommended the Symfony Routing component. There also exists the rather spiffy Symfony HttpFoundation component, which provides "an object-oriented layer for the HTTP specification." A word of warning: there is a PSR standard for representing HTTP messages (PSR-7 https://www.php-fig.org/psr/psr-7/), and Symfony HttpFoundation isn't strictly compliant with it. If this